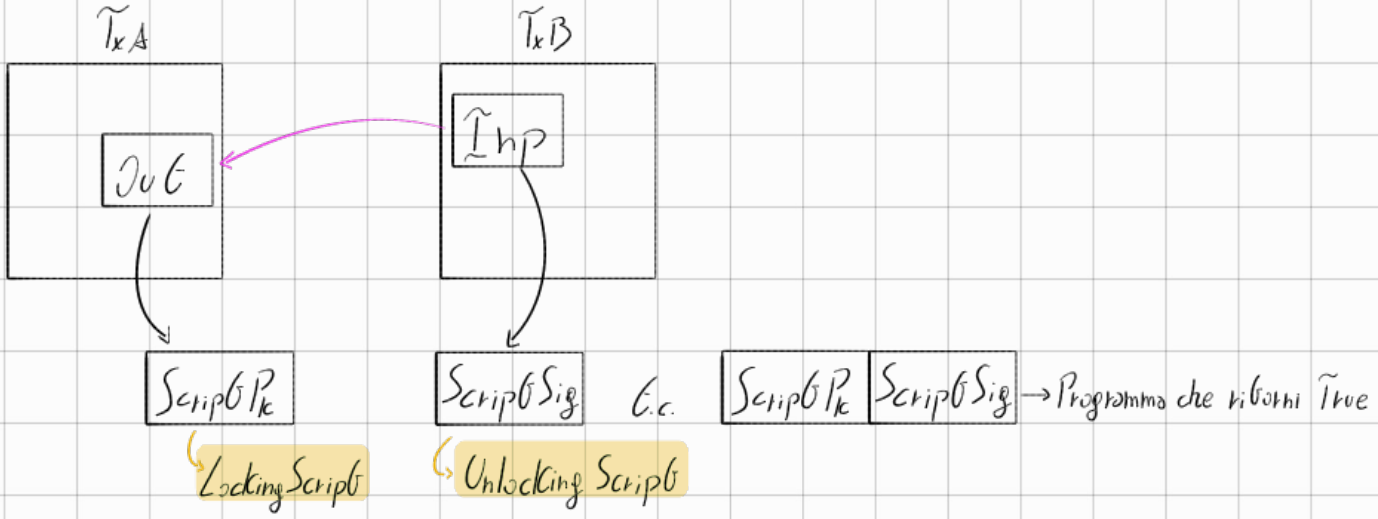
# Script

Abbiamo visto che le transazioni vengono validate mediante l'utilizzo di LockingScript e UnlockingScript

Questo meccanismo è stato progettato in modo tale che una transazione che voglia spendere un output debba contenere, all'interno di uno dei suoi input, un UnlockingScript t.c. la concatenazione tra UnlockingScript e LockingScript restituisca True




## Linguaggio Script

Il software del sistema Bitcoin fa in modo che all'interno di ogni nodo sia presente l'interprete del linguaggio Script

Questo linguaggio è

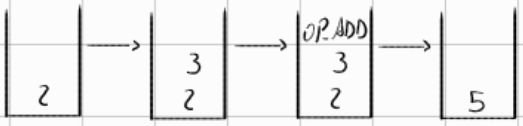
1. stateless → senza possibilità di utilizzare delle variabili
2. stack based → dati ed istruzioni vengono letti ed eseguiti come se fossero all'interno di una pila

Un programma è un'insieme di istruzioni e dati che vengono eseguiti su un modello stack based
Per quanto riguarda le istruzioni, queste possono estrarre ed immettere dati all'interno della pila

## Esempio

OP_ADD = prende i due elementi sottostanti della pila e ne calcola la somma

<2><3>OP_ADD =>



## Pay To Public Key (P2PK)

Il LockingScript più semplice è chiamato P2PK ed è formato da

<PK> OP_CHECKSIG → prende due dati e verifica se il primo sia la firma relativa al secondo

↳ chiave pubblica

Il relativo UnlockingScript sarà quindi della forma:

<sig> → chiave privata

In questo modo la concatenazione fra ScriptSig e ScriptPk è della forma

<sig><PK> OP_CHECKSIG =>




True se sig valida Pk
False altrimenti

Oss

Questa tipologia di script rende note a tutti le Publickey, per cui i bitcoin all'interno di quella transazione sono protetti "soltanto" dallo schema di firma digitale ECDSA

Se qualcuno riuscisse a rompere questo schema allora potrebbe spendere quei bitcoin

## Pay To Public Key Hash (P2PKH)

Per offrire un'ulteriore layer di sicurezza all'interno di questi script non compare direttamente la chiave pubblica, ma il suo hash.

OP_DUP OP_HASH160 <pkhash> OP_EqualVerify OP_CHECKSIG

↳ Esegue due valori e controlla se sono uguali

↳ Ne esegue un valore e immette l'hash

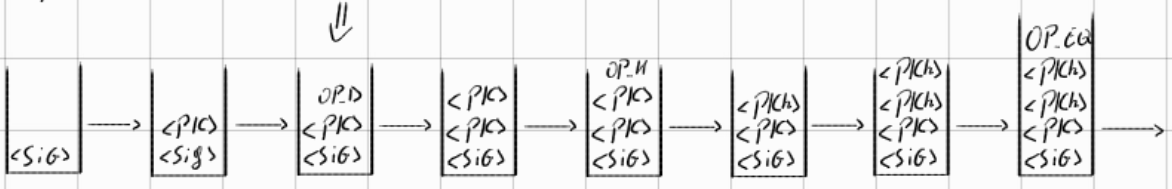
↳ Duplica l'elemento sottostante

Per questo ora all'interno del rispettivo UnlockingScript oltre alla <sig> deve essere presente anche <pk>

<sig> <pk>

In questo modo si ha:

<sig> <pk> OP_DUP OP_HASH160 <pkhash> OP_EqualVerify OP_CHECKSIG






Oss

Si può notare come la <pk> venga comunque resa pubblica nello ScriptSig,
Per questo di solito ogni coppia di chiavi viene usata una sola volta

## Pay To Multisig (P2m)

Può essere necessario a volte che l'output di una transazione possa essere speso solo se due o più persone si mettono d'accordo

c
2 <pkA> <pkB> <pkC> 3 OP_CHECKMULTISIG
n

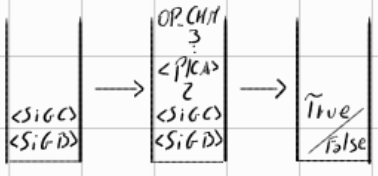
Prende un'intero n, n chiavi pubbliche, un'intero c, c firme e verifica che le firme siano valide rispetto alle n chiavi in input

Per cui il rispettivo Unlocking script sarà della forma

<SigB><SigC>

In questo modo si ha:

<SigB><SigC> 2 <pkA> <pkB> <pkC> 3 OP_CHECKMULTISIG




Oss

Ovviamente si può facilmente progettare uno Script di tipo MultiSig che permetta di usare l'hash delle Pk piuttosto che lasciarle in chiaro

## Pay To Script Hash (P2Sh)

All'interno dei precedenti schemi è il mittente che decide le condizioni con cui il destinatario possa spendere l'output

Per affidare questo compito al destinatario piuttosto che al mittente sono nati gli Script P2Sh dove è il destinatario che crea i redeem script e ne invia l'hash al mittente che lo inserisce nello ScriptPk

OP_HASH160 <redeem_script_hash> OP_EQUAL

Per cui il destinatario dovrà inserire all'interno dello ScriptSig il redeem Script seguito dalle condizioni necessarie per renderlo True

↳ Dopo aver verificato che l'hash del redeem corrisponde, questo viene eseguito

## Salvare dati nella blockchain

Essendo la blockchain di bitcoin un registro permanente ed immutabile, si è cominciato ad usare i bytes riservati agli scripts per salvare dei dati.

Unspent Transaction Output

Per evitare che questi vengano salvati all'interno dell'UTxO è stata introdotta l'istruzione OP_RETURN, che fa terminare qualsiasi script che la contenga.